Application Visual Studio C#

Phase 1 – Application backoffice du gérant d'hôtel

- **Mouad TOUNANI** – Développeur 1 (US1, US3, US5)
- **Dorian LORON MORRA** – Développeur 2 (US2, US4, US6)
- BTS SIO 2 SLAM
- 2026-2027

Sommaire

1. Expression des besoins....3
1.1 Contexte....3
1.2 Acteurs et utilisateurs....3
1.3 Objectifs....3
1.4 Fonctionnalités attendues....3

2. Conception / Spécifications techniques....4
2.1 Solution technique....4
2.2 Outils utilisés....4
2.3 Architecture....4
2.4 Données....5
2.5 Contraintes de réalisation....5
2.6 Conduite de projet....6
2.7 Maquettage / IHM....6

3. Développement....10
3.1 Réalisation fonctionnelle....10
3.2 Principes de développement respectés....10
3.3 Sécurité....10
3.4 Règles de nommage....11
3.5 Commentaires dans le code....11
3.6 Extraits de code....11

4. Exploitation / Tests....13
4.1 Stratégie de tests....13
4.2 Jeu d'essai....13
4.3 Validation multi-environnement....13
4.4 Résultats....13

5. Bilan....14
5.1 Retour projet....14
5.2 Compétences acquises....14
5.3 Difficultés rencontrées....14

1. Expression des besoins

1.1 Contexte

Brit Hotel est un réseau d'hôteliers indépendants qui compte 180 hôtels 2, 3 et 4 étoiles en France. Le groupe souhaite une solution informatique pour gérer ses hôtels, diffuser leurs informations auprès de la clientèle et gérer les réservations.

La phase 1 du projet consiste à développer l'**application backoffice** installée dans chaque hôtel. Elle permet au gérant d'administrer les données de son hôtel dans la base SQL Server centralisée, commune à tous les hôtels du groupe. Les phases suivantes (site web, application mobile) utiliseront la même base.

1.2 Acteurs et utilisateurs

- Demandeur : le groupe Brit Hotel (siège social à Paris).
- Utilisateur : le gestionnaire d'un hôtel, ici l'hôtel MACON - CENTRE (n°103).
- Product Owner : le professeur, auteur des user stories.
- Équipe Scrum : deux développeurs, Mouad TOUNANI (US1, US3, US5) et Dorian LORON MORRA (US2, US4, US6).

1.3 Objectifs

- Réserver l'accès au logiciel au gérant grâce à l'identifiant et au mot de passe de l'hôtel.
- Tenir à jour les informations, les équipements et les chambres de l'hôtel.
- Consulter les réservations de l'hôtel et celles d'un client.
- Créer et modifier les fiches clients.
- Protéger tous les mots de passe avec l'algorithme de hachage Argon2.

1.4 Fonctionnalités attendues

US	Fonctionnalité	Développeur
1	Se connecter avec l'identifiant et le mot de passe de l'hôtel pour accéder à l'écran d'accueil.	Dev 1
2	Visualiser et modifier les informations de l'hôtel (nom, adresse, téléphone, description, prix, mot de passe) et ses équipements.	Dev 2
3	Gérer les chambres : ajout et suppression, chaque chambre ayant un numéro propre à l'hôtel.	Dev 1
4	Visualiser les réservations de l'hôtel (dates, client, chambres) et les filtrer par date de début ou de fin.	Dev 2
5	À partir du n° d'un client, afficher son nom, son mail et toutes ses réservations (hôtel, dates, chambres).	Dev 1
6	Créer une fiche client et modifier celle d'un client existant (nom, mail, mot de passe).	Dev 2

2. Conception / Spécifications techniques

2.1 Solution technique

Application Windows Forms en C# (.NET Framework 4.8), développée en programmation orientée objet. L'accès aux données passe par **Entity Framework 6** (approche Database First) : le modèle Model1.edmx est généré à partir de la base SQL Server, et la classe de contexte **Bdd** donne accès aux tables sous forme de collections d'objets (maBdd.hotels, maBdd.chambres…) interrogées avec LINQ.

Les mots de passe des hôtels et des clients sont enregistrés sous forme d'empreinte **Argon2**, grâce au package NuGet Isopoh.Cryptography.Argon2.

2.2 Outils utilisés

- Visual Studio 2022 – C# / .NET Framework 4.8 – Windows Forms
- Entity Framework 6.2 (Database First)
- SQL Server 2019 et SQL Server Management Studio
- Isopoh.Cryptography.Argon2 (package NuGet)
- GitHub : dépôt emme-mouad/Hotel, une branche par développeur (travail-mouad, travail-dorian)
- Notion : organisation des tâches et suivi des user stories

2.3 Architecture



Le projet respecte l'architecture **MVC** demandée :

Couche	Rôle	Dans le projet
Modèle	Classes métier et accès aux données	Classes générées par Entity (hotel, chambre, client, reservation, equipement, photo) et contexte Bdd
Vue	Interface utilisateur	Fichiers Designer.cs des formulaires et classe Charte (charte graphique commune)
Contrôleur	Traitements déclenchés par l'utilisateur	Méthodes événementielles des formulaires (_Click, _Load, _CellEnter, _ValueChanged)

La classe **Varglob** conserve l'hôtel connecté (Varglob.monHotel) : tous les formulaires l'utilisent après la connexion pour n'afficher que les données de cet hôtel.

2.4 Données

La base bdhotel_mouaddorian a été créée à partir du script SQL fourni, puis corrigée avec le patch du professeur (mots de passe des hôtels 101 et 103) : chaque hôtel se connecte avec « password » suivi de son numéro. Diagramme de classes du modèle Entity :

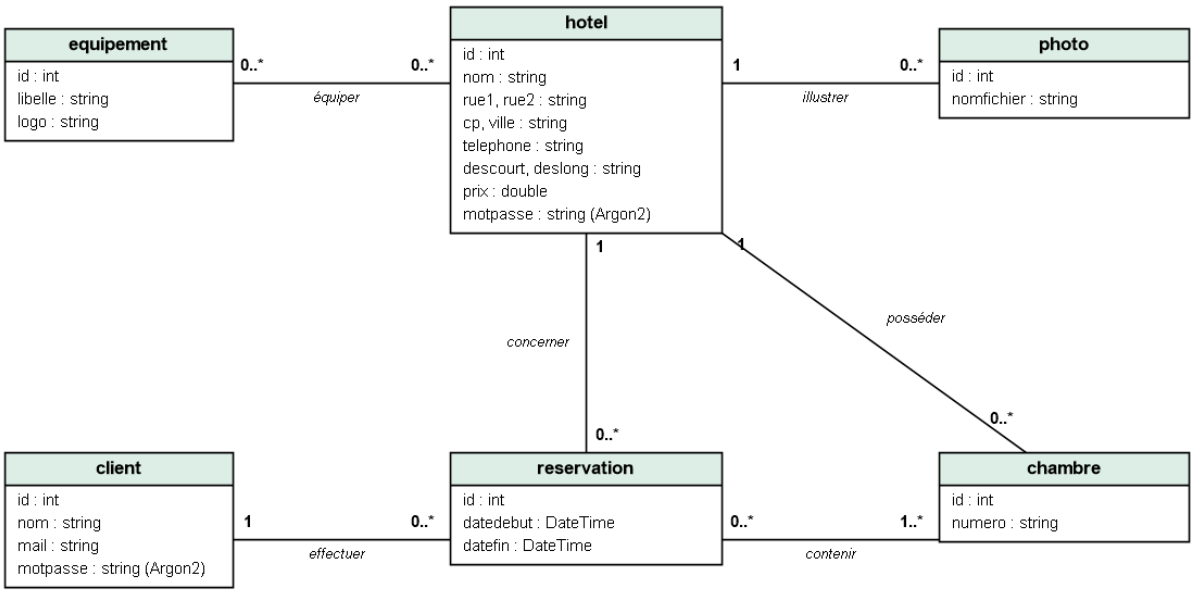


Schéma relationnel (clés étrangères précédées de #) :

```
hotel (id, nom, rue1, rue2, cp, ville, telephone, descourt, deslong, prix, motpasse)
chambre (id, numero, #hotel_id)
client (id, nom, mail, motpasse)
reservation (id, datedebut, datefin, #hotel_id, #client_id)
chambre_reservation (#reservation_id, #chambre_id)
equipement (id, libelle, logo)
equipement_hotel (#hotel_id, #equipement_id)
photo (id, nomfichier, #hotel_id)
```

2.5 Contraintes de réalisation

- Programmation orientée objet en C# avec Entity Framework, architecture MVC.
- Règles de nommage, commentaires réguliers et indentation.
- Mots de passe sécurisés par Argon2 : hachage à l'enregistrement, vérification à la connexion.
- Gestion des exceptions autour des accès à la base de données (try / catch).
- Chaîne de connexion fournie par le professeur, commune à tous les postes (App.config).
- Écrans homogènes respectant une même charte graphique.
- Base de données livrée propre, avec un jeu d'essai cohérent.

2.6 Conduite de projet

Le projet a été réalisé en équipe Scrum de deux développeurs. Chacun a développé ses user stories dans son propre projet Visual Studio, puis les formulaires ont été assemblés dans une même application. Les tâches ont été organisées sur Notion et le code est versionné sur GitHub, avec une branche par développeur.

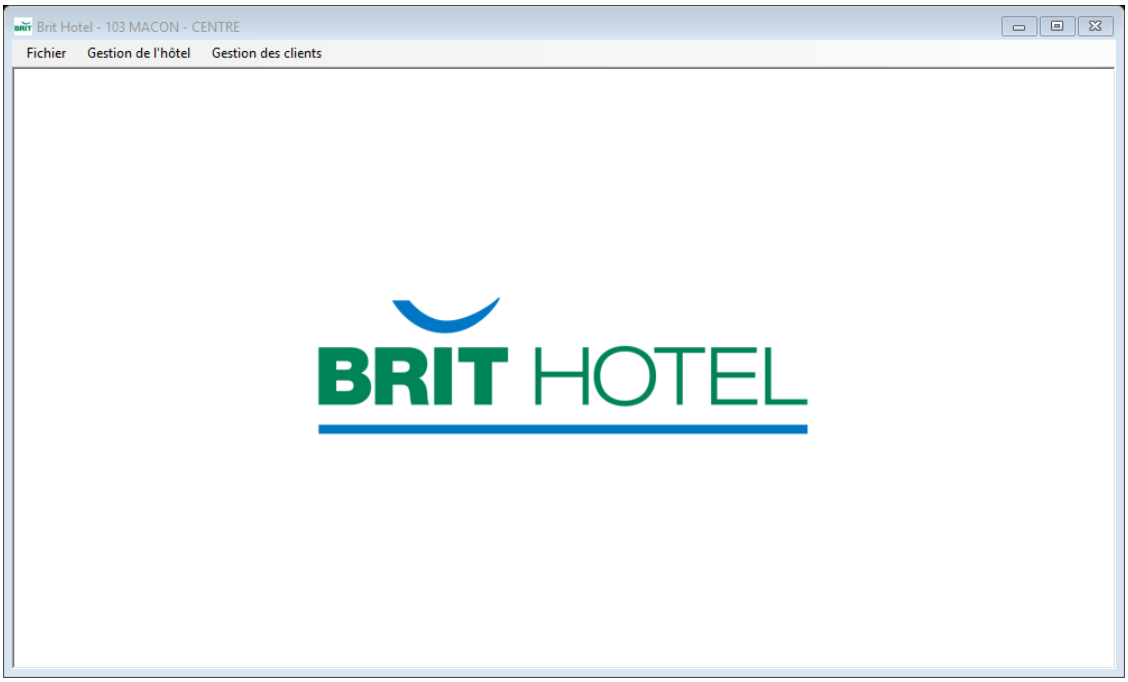
Date	Étape
25/09/2026	Création du dépôt GitHub et du projet Visual Studio (modèle Entity relié à la base).
29/09 – 23/10/2026	Développement en parallèle : US1, US3, US5 (Dev 1) et US2, US4, US6 (Dev 2, branche travail-dorian).
25/10/2026	Assemblage des formulaires dans le même projet (même namespace Hotel, « Ajouter un élément existant »), sauvegarde sur la branche travail-mouad.
25/10/2026	Tests et corrections de chaque user story : contrôles de saisie, hachage Argon2, gestion des exceptions.
25 – 28/10/2026	Jeu d'essai, cahier de recette, export du script SQL, charte graphique et vérification finale du code.
06/11/2026	Livraison : documentation PDF, projet Visual Studio et script SQL.

2.7 Maquettage / IHM

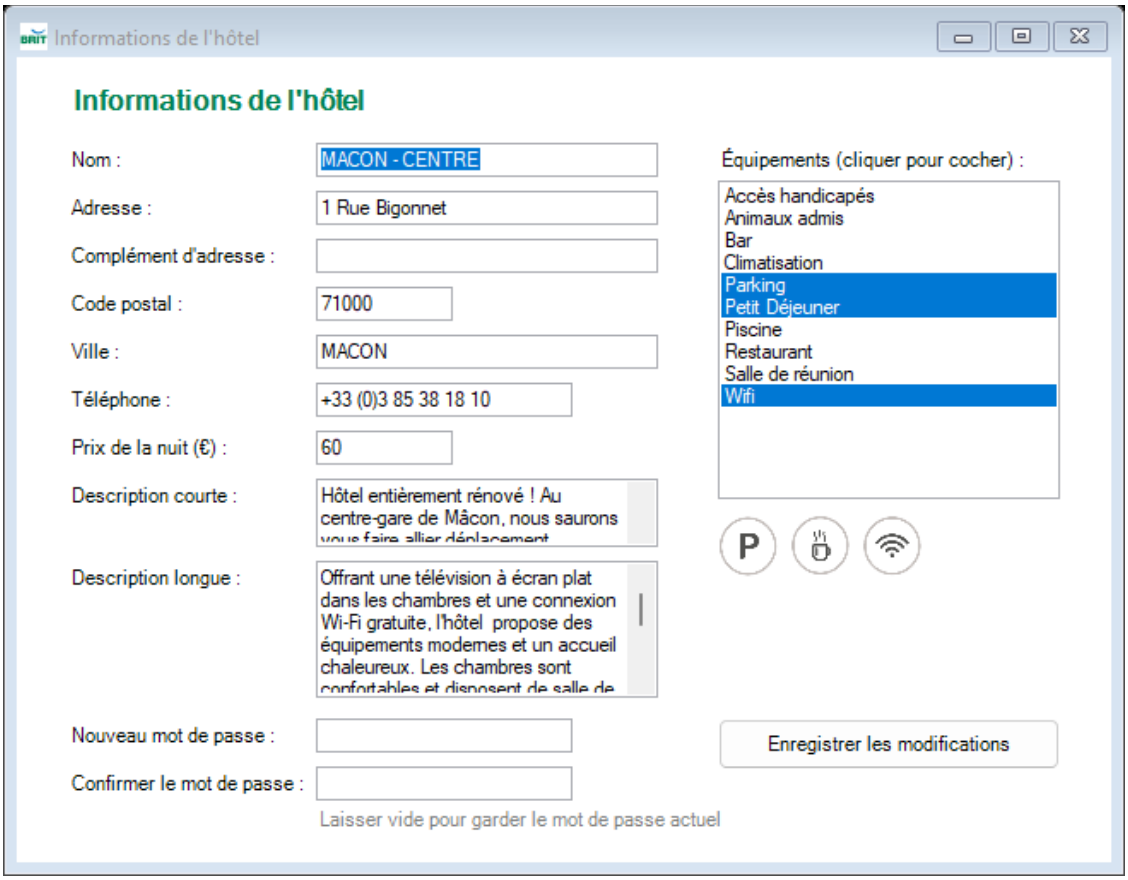
Connexion (US1) :



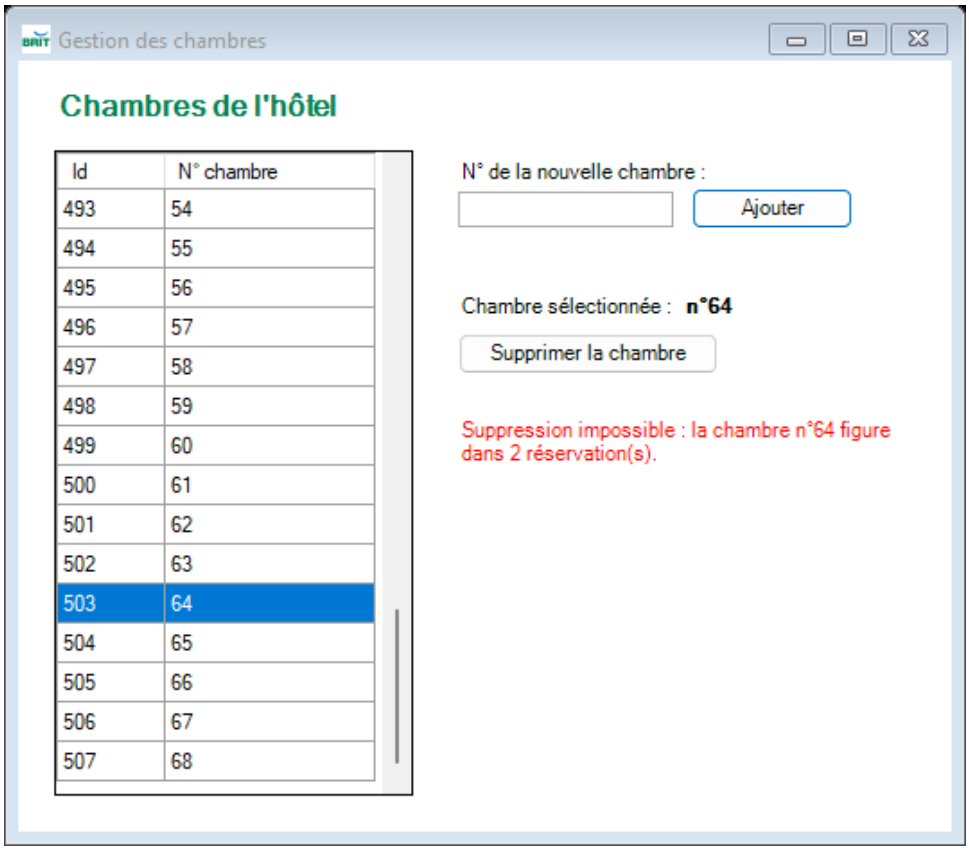
Écran d'accueil :



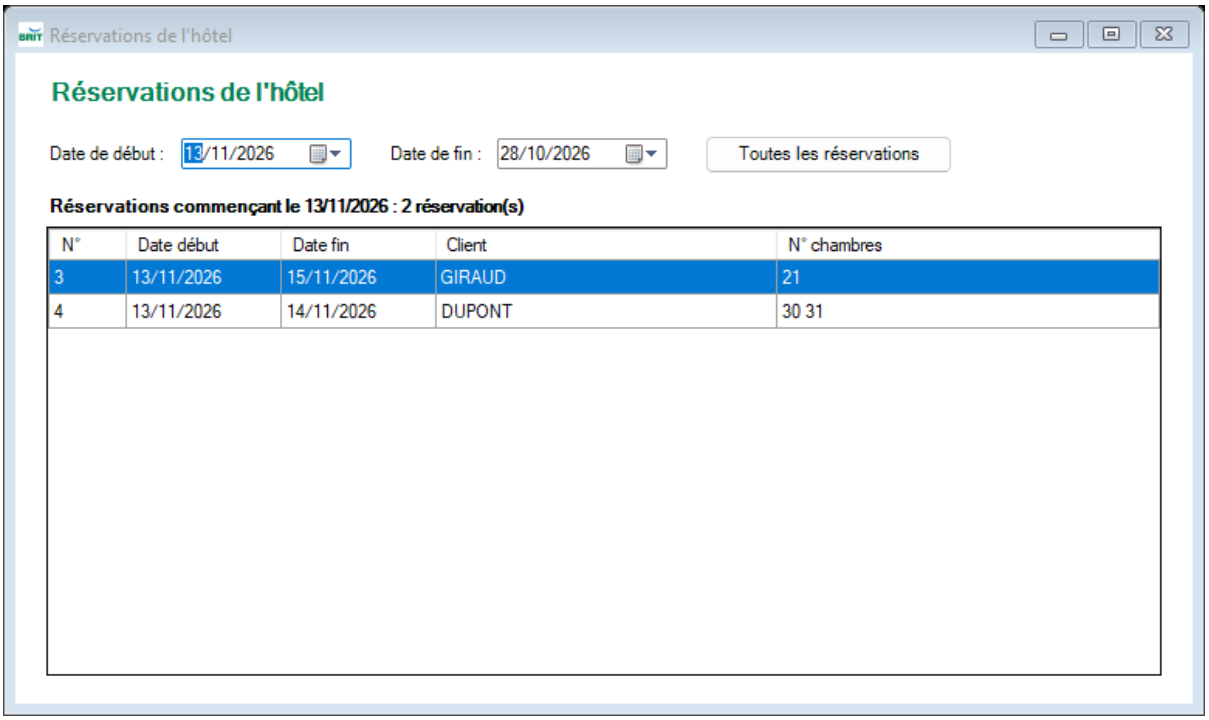
Informations de l'hôtel (US2) :



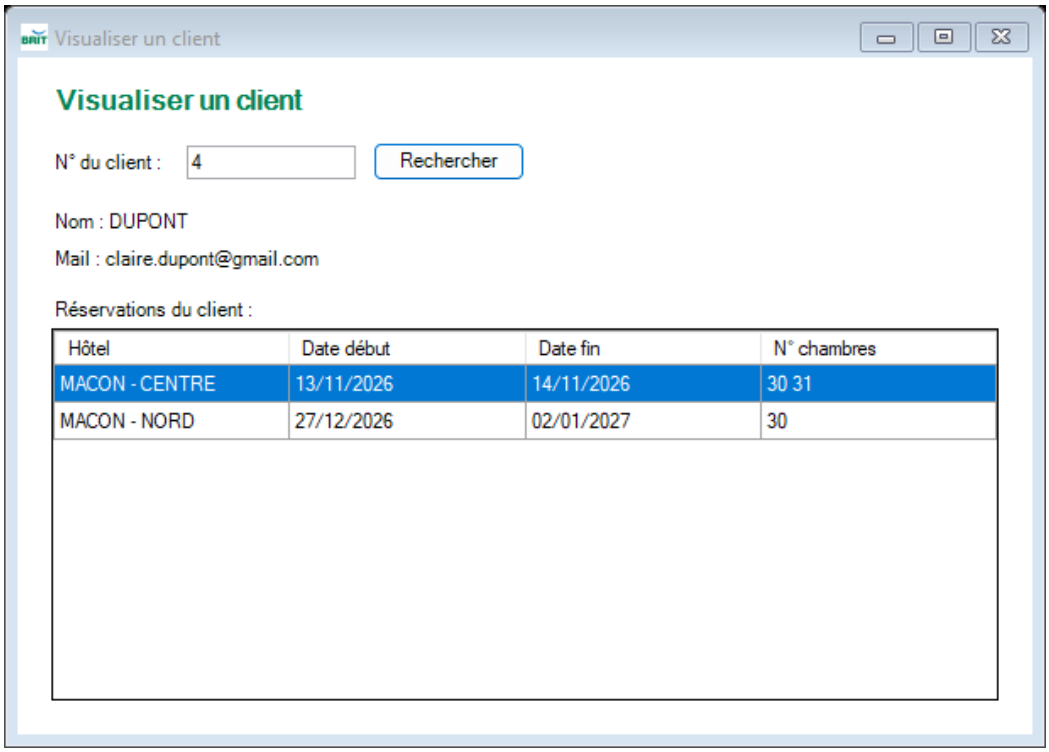
Gestion des chambres (US3) – tentative de suppression d'une chambre réservée :



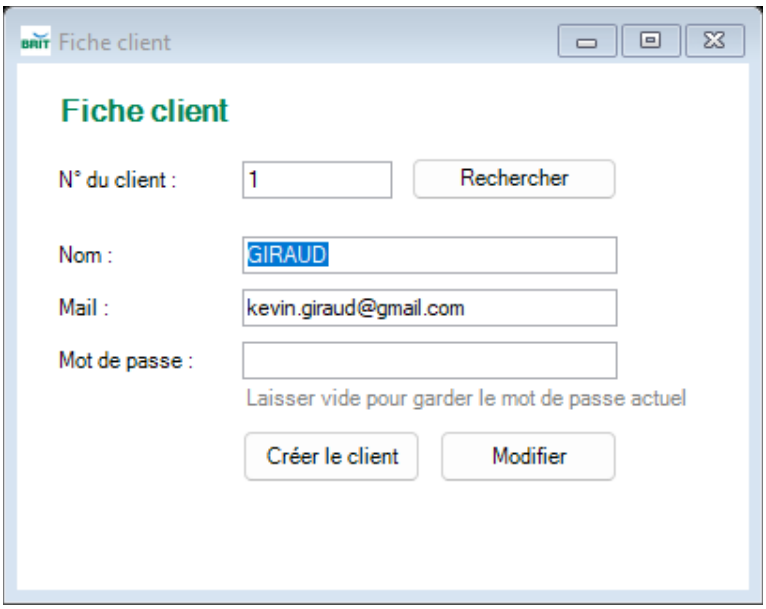
Réservations de l'hôtel (US4) – filtre sur la date de début :



Visualiser un client (US5) – client ayant réservé dans deux hôtels :



Fiche client (US6) :



3. Développement

3.1 Réalisation fonctionnelle

Les six user stories ont été implémentées, assemblées dans une même application et validées par le cahier de recette.

US	Formulaire	Points clés
1	FrmConnexion	Vérification avec Argon2.Verify, message d'erreur unique, mot de passe masqué, touche Entrée.
2	FrmHotel	Informations, équipements (liste à sélection multiple avec leurs logos) et nouveau mot de passe haché.
3	FrmChambres	Ajout avec contrôle des doublons ; suppression confirmée, refusée si la chambre est réservée.
4	FrmReservations	Tableau à 5 colonnes, filtre par date de début ou de fin (ValueChanged).
5	FrmVisualiserClients	Recherche par n° de client, réservations dans tous les hôtels du groupe.
6	FrmFicheClient	Création et modification, mot de passe haché, mail unique.
–	FrmAccueil	Menu ; formulaires ouverts à l'intérieur de l'accueil (MDI), un seul exemplaire par formulaire.

3.2 Principes de développement respectés

- Requêtes LINQ avec Entity Framework : aucune requête SQL écrite à la main, donc pas d'injection SQL.
- try / catch autour de chaque accès à la base : l'application ne s'arrête jamais, le message technique est écrit dans la fenêtre Sortie et l'utilisateur reçoit un message clair.
- Contrôles de saisie : champs vides, nombres (int.TryParse), longueurs maximales identiques à celles de la base, format du mail.
- Messages dans un label (rouge pour une erreur, vert pour une réussite) plutôt que des MessageBox, sauf pour confirmer une suppression.
- Données relues dans la base à chaque recherche, pour afficher les modifications faites dans une autre fenêtre.
- Prix lu au format français, quelle que soit la langue de Windows.

3.3 Sécurité

- Mots de passe hachés avec Argon2.Hash et vérifiés avec Argon2.Verify : aucun mot de passe en clair dans la base.
- L'empreinte n'est jamais affichée ; un mot de passe laissé vide n'est pas modifié.
- Nouveau mot de passe de l'hôtel saisi deux fois, 8 caractères minimum.
- À la connexion, le même message pour un identifiant ou un mot de passe faux.
- Suppression d'une chambre interdite si elle figure dans une réservation.
- Un même mail ne peut pas être utilisé par deux clients.

3.4 Règles de nommage

- Formulaires : préfixe Frm suivi du rôle (FrmConnexion, FrmChambres, FrmFicheClient…).
- Contrôles : préfixe selon le type – lbl, txt, btn, grd, dtp, lst, mnu, pic, flp (txtIdentifiant, grdChambres, dtpDebut, mnuQuitter…).
- Objets métier comme dans le TP Entity : maBdd, monHotel, leClient, laChambre, lesReservations, unEquipement.
- Classes du modèle : noms des tables (hotel, chambre, client…), collections au pluriel (chambres, reservations).
- Méthodes événementielles avec le suffixe standard (_Click, _Load, _CellEnter, _ValueChanged) ; méthodes internes nommées par un verbe (afficherChambres, afficherMessage).

Objectif : un code lisible et homogène entre les deux développeurs.

3.5 Commentaires dans le code

Comme le demandent les consignes de finalisation, le code est commenté toutes les 4 ou 5 lignes : intention de chaque bloc, accès à la base de données, contrôles de saisie et opérations sensibles (hachage, suppression).

3.6 Extraits de code

Connexion (US1) – vérification du mot de passe avec Argon2 :

```
try
{
    //Récupérer l'hôtel correspondant à l'identifiant saisi
    leHotel = maBdd.hotels.Where(param => param.id == identifiant).FirstOrDefault();

    //Comparer l'empreinte hash de la base avec la saisie en clair
    valide = false;
    if (leHotel != null && leHotel.motpasse != null)
    {
        valide = Argon2.Verify(leHotel.motpasse, txtMdp.Text);
    }

    if (valide == true)
    {
        //Mémoriser l'hôtel connecté pour tous les autres formulaires
        Varglob.monHotel = leHotel;
        //Ouvrir l'écran d'accueil et masquer le formulaire de connexion
        FrmAccueil accueil = new FrmAccueil();
        accueil.Show();
        this.Hide();
    }
    else
    {
        //Message volontairement commun pour ne pas indiquer lequel des deux est faux
        lblMessage.Text = "Identifiant ou mot de passe incorrect.";
        txtMdp.Clear();
        txtMdp.Focus();
    }
}
catch (Exception exception)
{
    //Erreur imprévue (serveur SQL inaccessible, ...) : l'application ne s'arrête pas
    Console.WriteLine(exception.Message);
    lblMessage.Text = "Connexion à la base de données impossible.";
}
```

Suppression d'une chambre (US3) :

```
private void btnSupprimer_Click(object sender, EventArgs e)
{
    //Vérifier qu'une chambre a été sélectionnée dans la liste
    if (laChambre == null)
    {
        afficherMessage("Sélectionnez d'abord une chambre dans la liste.", true);
        return;
    }

    try
    {
        //Une chambre déjà réservée ne peut pas être supprimée
        int nbReservations = laChambre.reservations.Count;
        if (nbReservations > 0)
        {
            afficherMessage("Suppression impossible : la chambre n°" + laChambre.numero
                            + " figure dans " + nbReservations + " réservation(s).", true);
            return;
        }

        //Demander confirmation car la suppression est définitive
        DialogResult reponse = MessageBox.Show(
            "Supprimer définitivement la chambre n°" + laChambre.numero + " ?",
            "Confirmation", MessageBoxButtons.YesNo, MessageBoxIcon.Question);
        if (reponse == DialogResult.Yes)
        {
            string numero = laChambre.numero;
            //Supprimer la chambre sélectionnée dans la base de données
            maBdd.chambres.Remove(laChambre);
            maBdd.SaveChanges();
            //Réafficher les chambres
            afficherChambres();
            afficherMessage("La chambre n°" + numero + " a été supprimée.", false);
        }
    }
    catch (Exception exception)
    {
        //Repartir d'une connexion propre et réafficher l'état réel de la base
        Console.WriteLine(exception.Message);
        maBdd = new Bdd();
        afficherChambres();
        afficherMessage("Erreur : la chambre n'a pas pu être supprimée.", true);
    }
}
```

Création d'un client (US6) – mot de passe haché avant l'enregistrement :

```
//Instancier le nouveau client avec son mot de passe haché par Argon2
client nouveauClient = new client();
nouveauClient.nom = nom;
nouveauClient.mail = mail;
nouveauClient.motpasse = Argon2.Hash(txtMotPasse.Text);

//Ajouter le client dans la base de données
maBdd.clients.Add(nouveauClient);
maBdd.SaveChanges();
```

4. Exploitation / Tests

4.1 Stratégie de tests

Un cahier de recette de 79 étapes a été rédigé, puis déroulé dans la peau d'un nouvel utilisateur. Chaque étape décrit l'action à réaliser et le résultat attendu. Il couvre :

- les cas nominaux de chaque user story ;
- les oublis et les saisies non prévues : lettres dans un numéro, prix négatif, mail invalide, mot de passe trop court ;
- les doublons : numéro de chambre, mail d'un client ;
- la suppression d'une chambre réservée et la confirmation de suppression ;
- la séparation des données entre deux hôtels (connexion avec l'hôtel 101) ;
- le comportement de l'application quand le serveur SQL est arrêté.

4.2 Jeu d'essai

La base fournie a été complétée par un jeu d'essai réaliste : 4 clients créés avec l'application (mots de passe hachés) et 7 réservations réparties sur les deux hôtels de Mâcon. La date de fin de la réservation n°2, antérieure à sa date de début, a été corrigée.

N°	Hôtel	Client	Début	Fin	Chambres
1	MACON - CENTRE	LENOIR	01/10/2026	04/10/2026	64 65 67
2	MACON - CENTRE	GIRAUD	02/10/2026	05/10/2026	12 14
3	MACON - CENTRE	GIRAUD	13/11/2026	15/11/2026	21
4	MACON - CENTRE	DUPONT	13/11/2026	14/11/2026	30 31
5	MACON - CENTRE	BERNARD	20/11/2026	22/11/2026	45
6	MACON - CENTRE	MOREAU	18/11/2026	22/11/2026	50 51 52
7	MACON - CENTRE	LENOIR	06/12/2026	08/12/2026	64
8	MACON - NORD	MERAD	13/11/2026	16/11/2026	20
9	MACON - NORD	DUPONT	27/12/2026	02/01/2027	30

Les mêmes dates de début ou de fin (13/11 et 22/11) permettent de vérifier le filtre de l'US4, et la cliente DUPONT a réservé dans deux hôtels pour illustrer l'US5.

4.3 Validation multi-environnement

L'application a été testée sur le poste de développement (Windows 11, SQL Server 2019) avec la chaîne de connexion fournie, commune à tous les postes. Le script SQL de livraison a été exécuté sur une base vierge : la base obtenue est identique à la base de développement (structure, données, collation French_CI_AS).

4.4 Résultats

Toutes les étapes du cahier de recette sont conformes au résultat attendu. Les six user stories sont opérationnelles et conformes au cahier des charges.

5. Bilan

5.1 Retour projet

Le projet a été mené en binôme avec une répartition claire des user stories. L'assemblage a demandé de la coordination : même namespace, mêmes conventions de nommage et relecture croisée des formulaires pour obtenir une application homogène.

5.2 Compétences acquises

- Développement d'une application C# WinForms avec Entity Framework (Database First, LINQ).
- Sécurisation des mots de passe avec Argon2.
- Travail en équipe Scrum et gestion des versions avec GitHub (une branche par développeur).
- Tests, gestion des exceptions et préparation d'une base de données de livraison.

5.3 Difficultés rencontrées

- **Assemblage des deux projets** : formulaires de test à retirer et conventions différentes. Résolu en important les formulaires dans le même namespace, puis en harmonisant les noms et la charte graphique.
- **Données périmées entre deux fenêtres** : chaque formulaire garde sa propre connexion Entity, qui conserve les données déjà lues ; un client renommé dans une fenêtre gardait son ancien nom dans une autre. Résolu en relisant la base à chaque recherche.
- **Format des nombres** : sur un poste configuré en italien, le prix « 59.90 » était lu 5990. Résolu en lisant le prix au format français.
- **Script SQL de livraison** : l'export contenait des chemins de fichiers propres à notre version de SQL Server. Résolu en simplifiant la création de la base, puis en testant le script sur une base vierge.